

FOCI projekt

Fejlesztői átadási dokumentáció

Állapot: 2026-03-31

Fókusz: backend + minimál működő frontend, zárt pilot előkészítve

Cél: új fejlesztő vagy új ChatGPT szál számára azonnal folytatható állapotkép

Backend	Frontend	Tesztek
Erős, service layeres, domain-szabályozott	Minimál, de működő pilot UI	5 suite / 12 teszt zöld

1. Projektcél és jelenlegi értelmezés

A FOCI projekt célja egy amatőr / baráti foci szervező rendszer felépítése, amely felhasználókat, csapatokat, csapattagokat, eseményeket, jelentkezéseket, várólistát és lemondás utáni automatikus előléptetést kezel. A projekt hosszabb távon döntéstámogató és csapatkiosztó irányba bővíthető, de a jelenlegi fókusz a stabil szervezési mag.

A projekt már túl van a puszta CRUD szinten. A backendben üzleti szabályok, állapotgép, jogosultsági réteg és service layer is ki lett alakítva. A frontend jelenleg minimál, de használható pilot felület.

2. Elkészült fő blokkok

2.1 Auth és felhasználói session

- JWT auth működik.
- Elkészült a POST /api/auth/register, POST /api/auth/login és GET /api/auth/me endpoint.
- A requireAuth middleware Bearer tokenból betölti a usert és req.user objektumot ad.
- A frontend képes session visszaépítésre, loginra, regisztrációra és logoutra.

2.2 Team domain

- Csapat létrehozás auth alapon működik.
- Csapat részletes lekérdezése működik.
- A captain automatikusan létrejön a team létrehozásakor.
- Elkészült a csapattag hozzáadás email alapján.
- Elkészült a csapattag szerepkör módosítás member / vice_captain szinten.
- Elkészült a csapattag eltávolítás soft módban (membership_status = inactive).

2.3 Event domain

- Esemény létrehozás csapathoz működik.
- Esemény részletes lekérdezése működik.
- Csapat eseményeinek listája működik.
- Esemény szerkesztés működik.
- Esemény státuszváltás működik.
- Elkészült az event state machine: draft, published, cancelled, finished.

2.4 Registration domain

- Jelentkezés eseményre működik.
- Lemondás működik.
- A státuszlogika going / waiting_list / cancelled alapon működik.
- Lemondás után az első várólistás automatikusan előlép.
- Újrajelentkezés lemondás után működik.

- Az aktív regisztrációkat DB oldali részleges egyedi védelem támogatja.

2.5 Saját csapatok / saját események

- Elkészült a GET /api/my/teams endpoint.
- Elkészült a GET /api/my/events endpoint.
- A frontend login után automatikusan képes saját csapatokat és saját eseményeket betölteni.
- A felhasználó már nem kizárólag team ID kézi megadásával tud dolgozni.

2.6 Service layer és hibatípusok

- A controllerek jelentős része service rétegbe lett kiszervezve.
- Elkészült az AppError mint üzleti hibahordozó.
- A dbService withTransaction mintával központosítja a tranzakciókezelést.
- A controllerek már vékonyabbak és főleg request/response adapter szerepet töltenek be.

3. Jelenlegi architektúra

A rendszer jelenleg Node.js + Express + PostgreSQL alapú backendből és egy Express által statikusan kiszolgált minimál frontendből áll. A frontend külön buildrendszer nélküli HTML / CSS / vanilla JS alapú pilot UI.

3.1 Fájlstruktúra

Terület	Fájlok
Frontend	public/index.html, public/styles.css, public/app.js
Controllers	authController.js, teamController.js, eventController.js, myController.js
Services	dbService.js, eventService.js, registrationService.js, teamService.js, myService.js
Routes	authRoutes.js, teamRoutes.js, eventRoutes.js, myRoutes.js
Middleware	requireAuth.js, teamPermissions.js
Utils	appError.js
Tests	auth.e2e.test.js, auth-register.e2e.test.js, event-state.e2e.test.js, event-registration.e2e.test.js, team-members.e2e.test.js

4. Fő endpointok

Terület	Metódus és útvonal	Megjegyzés
Auth	POST /api/auth/register	Regisztráció és azonnali token
Auth	POST /api/auth/login	Bejelentkezés
Auth	GET /api/auth/me	Aktuális user
Team	POST /api/teams	Csapat létrehozás
Team	GET /api/teams/:teamId	Csapat és taglista
Team	POST /api/teams/:teamId/members	Tag hozzáadása email alapján
Team	PATCH /api/teams/:teamId/members/:memberId	Szerepkör módosítás
Team	DELETE /api/teams/:teamId/members/:memberId	Soft eltávolítás
Event	POST /api/teams/:teamId/events	Esemény létrehozás
Event	GET /api/teams/:teamId/events	Csapat eseményei
Event	GET /api/events/:eventId	Esemény részletes nézete
Event	PATCH /api/events/:eventId	Esemény szerkesztés
Event	PATCH /api/events/:eventId/status	Státuszváltás
Reg.	POST /api/events/:eventId/register	Jelentkezés
Reg.	POST /api/events/:eventId/cancel	Lemondás
Dashboard	GET /api/my/teams	Saját aktív csapatok
Dashboard	GET /api/my/events	Saját csapatok eseményei

5. Fontos üzleti szabályok

5.1 Event state machine

- Státuszok: draft, published, cancelled, finished.

- Draft esemény teljesen szerkeszthető.
- Published esemény csak soft mezőkkel szerkeszthető.
- Cancelled és finished esemény nem szerkeszthető.
- Tiltott státuszátmenetek blokkolva vannak.

5.2 Registration szabályok

- maxPlayers számolása: `playersOnFieldTotal + substitutesCount`.
- Ha van hely, a státusz going.
- Ha nincs hely, a státusz waiting_list.
- Ha egy going user lemond, az első waiting_list user automatikusan going lesz.
- A user nem lehet egyszerre többször aktív jelentkező ugyanarra az eseményre.

5.3 Team member szabályok

- Tag hozzáadás email alapján történik, csak captain vagy vice_captain jogosultsággal.
- A captain ezen a felületen nem módosítható és nem távolítható el.
- A szerepkör itt csak member vagy vice_captain lehet.
- Eltávolítás soft módon történik: `membership_status = inactive`.

6. Frontend jelenlegi működése

A frontend egy minimál, de működő pilot UI. Nem végleges design, hanem funkcionális integrációs réteg.

- Login és register oldal működik.
- Session visszaépítés működik localStorage alapján.
- Logout működik.
- Admin oldalon működik: csapat létrehozás, csapat betöltés, tagkezelés, esemény létrehozás, esemény szerkesztés, státuszváltás.
- User oldalon működik: saját csapatok, saját események, csapat eseményeinek listája, esemény részletei, jelentkezés, lemondás.
- A public/app.js legutóbb kézzel javítva lett, hogy a saját csapatok és saját események automatikusan betöltődjenek, a dashboard gombok jó függvényeket hívjanak, és a member-management stabil legyen.

7. Tesztelés

Jelenlegi biztos állapot szerint a tesztek zöldek voltak: 5 suite, 12 teszt.

- `auth.e2e.test.js`
- `auth-register.e2e.test.js`
- `event-state.e2e.test.js`
- `event-registration.e2e.test.js`

- team-members.e2e.test.js

Ajánlott következő ellenőrzés: a legfrissebb my dashboard backend + public/app.js módosítások után teljes npm test újrafuttatás, hogy a dashboard réteg is explicit regresszióvédelem alá kerüljön.

8. Jelenlegi hiányok és nyitott kérdések

- Nincs captain transfer / ownership átadás flow.
- Nincs valódi invite + accept vagy self-join mechanizmus.
- Nincs password reset.
- Nincs rate limiting / brute force védelem teljesen lezárva.
- Nincs production hardening, monitoring, backup, staging stratégia teljesen készre húzva.
- A frontend használható, de még minimál pilot szintű.

9. Javasolt következő lépések

- Captain transfer flow kialakítása.
- Invite/join flow kialakítása.
- Frontend UX finomítás: role-based gombok, jobb empty state-ek, finomabb hibakezelés.
- A my dashboard rétegre külön E2E suite bevezetése, ha még nincs lezárva.
- Pilot seed adatok és staging környezet.

10. Fejlesztői indulási pont új kéz számára

Ha egy új fejlesztő veszi át a projektet, a legfontosabb tudnivaló az, hogy a rendszer magja már erős. Nem auth-normalizálást vagy alap event-logikát kell újraépíteni, hanem a hiányzó termékfolyamatokat kell lezárni. A legjobb következő feature jelen állapotban a captain transfer és/vagy invite/join flow.

11. Következő chatbe másolható rövid átdó

FOCI projektet folytatjuk. Ez már a 3. külön chat a projekten belül.

Jelenlegi állapot röviden:

- Node.js + Express + PostgreSQL backend
- JWT auth működik
- register/login/me kész
- team create/read kész
- team member add / role update / remove kész
- event create/read/update/status kész
- event state machine kész

- register / cancel / waiting_list / auto-promotion kész
- service layer refaktor kész
- minimál frontend van public/index.html + public/app.js alapon
- frontendből megy login, register, csapatbetöltés, admin event management, user event nézet
- elkészült a /api/my/teams és /api/my/events
- a public/app.js javított változata kezeli a session visszaépítést, saját csapatok/események automatikus betöltését és team member admin flow-t

Fő hiányok:

- nincs captain transfer flow
- nincs invite/accept vagy self-join flow
- nincs password reset
- nincs production hardening

A helyes következő fejlesztési lépés:

1. captain transfer vagy invite/join flow
2. frontend UX finomítás
3. teljes friss tesztkör újrafuttatás
4. pilot readiness

Innen folytassuk, ne a régi auth-normalizálási problémáktól.